

Update Manual for `phase_velocity.py`

RMFsolver package update notes

April 10, 2026

1 Purpose of This Manual

This document records the recent steady-front updates implemented in `RMFsolver/phase_velocity.py`. The recommended current production path is the compact-coordinate boundary-value solver

```
solve_steady_front_1d_aQstar_rescale_bvp(...).
```

This solver fixes the interface composition a_{Q^*} , solves for the compatible baryon flux j_B , and enforces the interface and downstream stable tail conditions in a single BVP solve.

Older one-dimensional shooting and compact-IVP notes are kept in the $\text{\LaTeX}$ source for history, but they are wrapped in an `\iffalse ... \fi` block. The rendered manual now focuses on the latest BVP workflow and the helper functions needed to reuse it elsewhere.

The emphasis here is on:

- the current BVP method and why it replaced the intermediate shooting paths,
- how the shared helper functions construct the nuclear, interface, and quark endpoint states,
- how local hydro closure is performed without assuming constant quark-side velocity,
- how branch selection is handled for the μ_K -rich and μ_K -poor solutions,
- how to configure scan scripts so the BVP stays on the physical branch.

2 High-Level Summary of the Update

The steady-front portion of `phase_velocity.py` now contains several historical solver paths, but the current recommended path is `solve_steady_front_1d_aQstar_rescale_bvp(...)`. It treats a_{Q^*} as the scan variable and solves for j_B together with the functions $a(s)$ and $q(s)$ using `scipy.integrate.solve_bvp`.

The older solver paths are still present in the Python file for comparison and regression checks:

- `solve_steady_front_2d(...)`: historical 2D shooting in (j_B, a_{Q^*}) ;
- `solve_steady_front_1d_aQstar(...)`: finite-domain scalar shooting in j_B at fixed a_{Q^*} ;
- `solve_steady_front_1d_aQstar_rescaled(...)`: compact-coordinate IVP shooting in j_B .

Those intermediate 1D methods are archived in the source of this manual but are not rendered, so the current document can serve as a clean reference for the BVP implementation.

3 Shared Physics Model Used by Both Solvers

3.1 Steady equations on the quark side

On the quark side ($x > 0$) the code solves the steady system

$$j_B = u(x) n_B(x), \quad (1)$$

$$\Pi = h(x) u(x)^2 + P(x), \quad (2)$$

$$\partial_x (D \partial_x a - ua) = R(a), \quad (3)$$

where the composition variable is

$$a = \frac{n_K - n_K^Q}{n_B^Q}. \quad (4)$$

The flux variable is defined as

$$q(x) = D \partial_x a - u(x) a(x). \quad (5)$$

The first-order system actually integrated by the code is

$$\frac{da}{dx} = \frac{q + ua}{D}, \quad (6)$$

$$\frac{dq}{dx} = \gamma (a^3 + \eta a). \quad (7)$$

3.2 Endpoint-state notation

The code uses the following notation consistently:

- **N**: upstream nuclear state at $x = 0^-$,
- Q^* : immediate quark-side interface state at $x = 0^+$,
- **Q**: far-right equilibrated quark state as $x \rightarrow \infty$.

3.3 Frozen microphysics convention

Both steady-front solvers use the same convention:

- D , η , and γ are evaluated at Q^* ,
- they are then kept fixed during the IVP integration.

4 New and Modified Helper Functions

4.1 `nK_QM(muB, muK, B_one_forth, T, ms=0, upB=5000, dmu=None)`

This helper returns the kaon-density-like thermodynamic derivative

$$n_K = \frac{\partial P_{QM}}{\partial \mu_K}.$$

In the current implementation it is handled analytically from the quark flavor densities:

$$n_K = \frac{n_d - n_s}{2}. \quad (8)$$

The function calls the existing quark-density helper, obtains the physical flavor densities n_d and n_s , and returns their half-difference. This is cleaner than the earlier finite-difference pressure derivative and avoids introducing extra numerical noise into the steady-front state construction.

4.2 `_solve_muB_Q_at_muK0_for_given_Pi(...)`

This helper solves for the far-right equilibrated quark endpoint Q at

$$\mu_K = 0$$

for a given momentum flux Π and baryon flux j_B .

Numerically it solves

$$\Pi_{QM}(\mu_B, 0, B^{1/4}, T, j_B) - \Pi = 0 \quad (9)$$

using `fsolve` with an initial guess of 1100 MeV. After obtaining the root, it checks that the corresponding quark baryon density is positive.

4.3 `_branch_muK_seed(a_like)`

This helper returns the positive- μ_K seed used by the current steady-front quark-state solves:

$$\mu_K^{\text{seed}} = \max(1, 20, 250|a|), \quad (10)$$

in MeV. This seed is used both at Q^* and during local continuation along the profile. The current code does not implement a separate near-zero μ_K branch mode in this helper.

4.4 `_solve_interface_Qstar_from_aQstar_and_Pi(...)`

This helper solves for the interface state Q^* by determining $(\mu_B^{Q^*}, \mu_K^{Q^*})$ from the pair of equations

$$\Pi_{QM}(\mu_B, \mu_K, B^{1/4}, T, j_B) = \Pi, \quad (11)$$

$$\frac{n_K(\mu_B, \mu_K) - n_K^Q}{n_B^Q} = a_{Q^*}. \quad (12)$$

The implementation details are:

1. It first builds a list of initial guesses.
2. These guesses are all positive- μ_K seeds and include stronger positive guesses for larger $|a_{Q^*}|$.
3. For each guess, it calls `scipy.optimize.root(..., method="hybr")`.
4. Successful roots are filtered to require positive density and nonnegative μ_K .
5. Near-duplicate roots are deduplicated.
6. The remaining candidates are ranked by closeness to the positive- μ_K target seed and then by larger μ_K , so that the solver stays on the intended positive- μ_K physical branch.

This helper is central to maintaining branch consistency in the steady-front machinery.

4.5 `_solve_local_quark_state_from_a_and_Pi(...)`

This helper solves the local quark thermodynamic state at a given point in the profile. Its unknowns are again (μ_B, μ_K) , but now the composition constraint uses the local $a(x)$ instead of a_{Q^*} .

It reuses `_solve_interface_Qstar_from_aQstar_and_Pi(...)` with $a \leftrightarrow a_{Q^*}$, then computes

$$n_B(x), \quad u(x) = \frac{j_B}{n_B(x)}. \quad (13)$$

In both the 2D and 1D solvers, the previous successful local state is passed as the next initial guess. That gives the code a nearest-neighbor continuation strategy along the profile and helps reduce accidental branch switching.

4.6 `_microphysics_at_Qstar(muB_Qstar, T)`

This helper computes the frozen microphysics coefficients using the current implementation convention:

$$\mu_{Q^*} = \mu_B^{Q^*}/3, \quad (14)$$

$$q_D = \sqrt{\frac{3g_s^2(\mu_{Q^*})^2}{2\pi^2}}, \quad (15)$$

$$D = \left[\frac{24\alpha_s^2}{\pi} \left(h_{\text{const}} \frac{T^{5/3}}{q_D^{2/3}} + \frac{\pi^3 T^2}{12q_D} \right) \right]^{-1}, \quad (16)$$

$$\eta = \frac{9\pi^2 T^2}{(\mu_{Q^*})^2}, \quad (17)$$

$$\tau = 1.98 \times 10^{12} \left(\frac{300}{\mu_{Q^*}} \right)^5, \quad (18)$$

$$\gamma = \frac{1}{\tau}. \quad (19)$$

These are returned in a dictionary and then treated as constants during the IVP.

5 Behavior of the 2D Steady-Front Solver

5.1 Function

`solve_steady_front_2d(...)`

5.2 Unknowns and residual

The 2D solver still solves for both

$$j_B, \quad a_{Q^*}$$

using the outer shooting residual

$$(a(x_{\text{max}}), q(x_{\text{max}})).$$

Internally it solves in variables

$$(\log j_B, a_{Q^*})$$

so that positivity of j_B is automatic.

5.3 Shared construction path

For each shooting trial it:

1. builds the upstream nuclear state N ,
2. solves the far-right equilibrated quark state Q at $\mu_K = 0$,
3. sets

$$n_K^Q = 0$$

exactly whenever $|m_s| \leq 10^{-12}$,

4. computes

$$a_N = \frac{n_B^N - n_K^Q}{n_B^Q},$$

5. solves the interface state Q^* using positive- μ_K candidate-ranked root selection,
6. computes frozen (D, η, γ, τ) at Q^* ,
7. sets the initial condition

$$q(0^+) = -a_N u_N,$$

8. integrates the IVP on $[0, x_{\max}]$.

5.4 Trivial-branch rejection

The 2D solver also contains a guard against a near-trivial branch where:

- $a(0^+)$ is numerically tiny,
- $q(0^+)$ is numerically tiny,
- Q^* nearly collapses onto the equilibrated state Q .

6 Recommended Solver: Compact-Coordinate 1D BVP

6.1 Function and role

The recommended current solver is

```
solve_steady_front_1d_aQstar_rescale_bvp(...).
```

It is the cleanest implementation of the steady hydro–diffusion–reaction front because it solves the front profile and the baryon flux j_B together as a boundary-value problem. The input a_{Q^*} is treated as the continuation or scan parameter; j_B is the unknown scalar parameter adjusted by the BVP solver.

The function is designed as a drop-in solver for other scripts: provide the physical inputs $(T, n_B^N, B^{1/4})$, the fixed interface composition a_{Q^*} , and optionally the branch switch `muK_rich`. The return value is a dictionary containing the endpoint states, the BVP diagnostics, and, if requested, the full profile arrays.

6.2 Inputs that define the physical problem

The primary inputs are:

- **T**: temperature;
- **nB_N**: upstream nuclear baryon density at 0^- ;
- **B_one_forth**: bag-model parameter $B^{1/4}$;
- **aQstar**: prescribed interface composition $a(0^+)$;
- **ms**: strange-quark mass, with production scans usually using $m_s = 0$;
- **muK_rich**: explicit branch selector. **True** follows the strange-diffusion / weak-disequilibrium branch. **False** follows the near- $\mu_K = 0$ branch.

Numerical controls include **tail_eps**, **n_mesh**, **tol_bvp**, **max_nodes**, **jB_guess**, **jB_bounds**, and **return_profile**.

6.3 What the solver borrows from the old numerical code

The old `_vNtoQ_num(...)` family used a BVP solve and a compact coordinate. The new solver borrows only that numerical architecture. It does *not* copy the old physical simplifications. In particular, it does not assume a constant quark-side velocity and it does not remove baryon-flux conservation.

The current equations remain

$$j_B = u(x)n_B(x), \quad (20)$$

$$\Pi = h(x)u(x)^2 + P(x), \quad (21)$$

$$\frac{da}{dx} = \frac{q + ua}{D}, \quad (22)$$

$$\frac{dq}{dx} = \gamma(a^3 + \eta a), \quad (23)$$

with the local velocity always reconstructed from

$$u(x) = \frac{j_B}{n_B(x)}. \quad (24)$$

6.4 Compact coordinate and transformed ODE

The compact coordinate is

$$s = 1 - e^{-x/\kappa}, \quad 0 \leq s < 1, \quad (25)$$

so that

$$x(s) = -\kappa \log(1 - s), \quad \frac{dx}{ds} = \frac{\kappa}{1 - s}. \quad (26)$$

The BVP endpoint is truncated at

$$s_{\text{end}} = 1 - \text{tail_eps}. \quad (27)$$

The scale κ is derived from the downstream linear tail:

$$u_Q = \frac{j_B}{n_B^Q}, \quad (28)$$

$$\lambda = \frac{-u_Q + \sqrt{u_Q^2 + 4D\gamma\eta}}{2D}, \quad (29)$$

$$\kappa = \frac{\text{kappa_factor}}{\lambda}. \quad (30)$$

Here u_Q and n_B^Q are evaluated at the equilibrated far-right quark state Q , while D , η , and γ are frozen at the interface state Q^* .

By the chain rule the equations supplied to `solve_bvp` are

$$\frac{da}{ds} = \frac{q + ua}{D} \frac{\kappa}{1 - s}, \quad (31)$$

$$\frac{dq}{ds} = \gamma(a^3 + \eta a) \frac{\kappa}{1 - s}. \quad (32)$$

This is only a coordinate change; the physics and closure are unchanged.

6.5 Unknown BVP parameter and bounds

The unknown BVP parameter controls j_B . If no explicit bounds are supplied,

$$j_B = e^\theta, \quad (33)$$

which guarantees $j_B > 0$. If `jB_bounds=(j_min,j_max)` is supplied, the code uses the bounded map

$$j_B = j_{\min} + (j_{\max} - j_{\min}) \frac{1}{1 + e^{-\theta}}. \quad (34)$$

For stable scans, the upper bound should not be set absurdly high. In the 6Apr runs, allowing `jB_max=50` let the BVP escape to an unphysical upper boundary; the conservative scan default is now `jB_bounds=1e-5,5.0`.

6.6 Endpoint-state construction helper chain

For each trial value of j_B , the solver constructs the global state as follows:

1. Compute the upstream nuclear state N from (T, n_B^N) using `PNM_n(...)` and `edensNM_n(...)`. Then

$$u_N = \frac{j_B}{n_B^N}, \quad \Pi = h_N u_N^2 + P_N.$$

2. Use `_solve_muB_Q_at_muK0_for_given_Pi(...)` to find the downstream equilibrated quark state Q at $\mu_K = 0$.
3. Compute n_B^Q with `nB_QM(...)` and n_K^Q with `nK_QM(...)`. For $m_s = 0$, the code sets $n_K^Q = 0$ exactly by symmetry at $\mu_K = 0$.
4. Compute

$$a_N = \frac{n_B^N - n_K^Q}{n_B^Q}.$$

5. Use the interface-state helper `_solve_interface_Qstar_from_aQstar_and_Pi(...)` to find $(\mu_B^{Q^*}, \mu_K^{Q^*})$ from momentum-flux matching and the fixed composition condition

$$\frac{n_K(\mu_B, \mu_K) - n_K^Q}{n_B^Q} = a_{Q^*}.$$

6. Use `_microphysics_at_Qstar(...)` to compute the frozen coefficients D , η , γ , and τ .

This helper chain is the reusable part of the implementation: any external copy of the method should keep this state-construction order unchanged.

6.7 Local hydro closure inside the BVP

At each BVP mesh point, the local profile value $a(s)$ is known but the quark thermodynamic state is not. The solver calls `_solve_local_quark_state_from_a_and_Pi(...)` to solve

$$\Pi_{QM}(\mu_B, \mu_K; j_B) - \Pi = 0, \quad (35)$$

$$\frac{n_K(\mu_B, \mu_K) - n_K^Q}{n_B^Q} - a(s) = 0. \quad (36)$$

Then it computes

$$n_B(s) = n_B^{QM}(\mu_B, \mu_K), \quad u(s) = \frac{j_B}{n_B(s)}. \quad (37)$$

The previous mesh-point solution is used as the next initial guess whenever possible, preserving nearest-neighbor continuation and reducing accidental branch jumps.

6.8 Boundary conditions

Because the system has two first-order ODE variables and one unknown scalar parameter, the BVP needs three boundary conditions:

$$a(0) - a_{Q^*} = 0, \quad (38)$$

$$q(0) + a_N u_N = 0, \quad (39)$$

$$q(s_{\text{end}}) + (D\lambda + u_Q)a(s_{\text{end}}) = 0. \quad (40)$$

The first fixes the input interface composition. The second is the interface flux jump condition

$$q(0^+) = -a_N u_N.$$

The third enforces the downstream stable-tail manifold.

6.9 Physical branch convention

The current steady-front implementation does not expose a `muK_rich=True/False` switch or a deliberate μ_K -poor mode. The helper `_branch_muK_seed(...)` supplies only positive- μ_K seeds, and the interface/local closure helpers rank candidate roots against those positive seeds. In other words, the code follows a single positive- μ_K physical branch. The returned string `branch_label="muK-rich"` is retained only as a compatibility label in result dictionaries.

6.10 Returned dictionary

The BVP solver returns a dictionary containing, at minimum, `success`, `message`, `jB`, `aQstar`, `a_N`, `u_N`, `u_Q`, `Pi`, `muB_Q`, `nB_Q`, `nK_Q`, `muB_Qstar`, `muK_Qstar`, `nB_Qstar`, `D`, `eta`, `gamma`, `tau`, `lambda`, `kappa`, `s_end`, `x_end`, `tail_residual`, `tail_residual_norm`, and BVP diagnostics such as `bvp_status`, `bvp_message`, `bvp_niter`, and `bvp_nodes`.

If `return_profile=True`, it also returns arrays `s`, `x`, `a`, `q`, `u`, `nB`, `muB`, and `muK`.

6.11 Recommended scan usage

The 6Apr scan driver should follow the branch by continuation rather than starting each a_{Q^*} in an isolated worker. The conservative recommended command is

```
python steady_front_scan.py \  
  --workers 1 \  
  --chunk-size 20 \  
  --jB-bounds 1e-5,5.0 \  
  --jB-aQstar-slope 1.2 \  
  --tail-eps 1e-8 \  
  --verb-mode simple
```

The scan script now solves the scalar BVP first and only reconstructs profiles after scalar success. This prevents a late profile-reconstruction failure from hiding an otherwise useful scalar result for j_B .

7 Entropy-Enabled Compact BVP Solver

7.1 Purpose and relation to the older compact BVP

The function `solve_steady_front_1d_aQstar_rescale_bvp(...)` remains the non-entropy compact-coordinate BVP path. It solves for (a, q) together with the unknown scalar flux j_B , uses the fixed temperature input T , and freezes D , η , and γ at the interface state Q^* .

The newer function `solve_steady_front_entropy(...)` keeps the same overall compact-coordinate architecture but enlarges the state to include entropy production and a variable temperature profile. The ODE variables are

$$a(x), \quad q(x), \quad w(x) = s(x)u(x), \quad (41)$$

and the local thermodynamic closure is upgraded from (μ_B, μ_K) to $(\mu_B, \mu_K, \log T)$. In this entropy-enabled path the temperature is reconstructed pointwise, and the microphysics coefficients are evaluated locally rather than frozen once at Q^* .

7.2 Helper chain and solver workflow

The entropy-enabled implementation is organized as a reusable helper chain:

1. `_quark_thermo_state(...)` builds a fully `ms`-consistent local quark state from (μ_B, μ_K, T, j_B) and returns $n_B, n_K, P, e, h, u, s, w = su$, and Π .
2. `_solve_muB_Q_at_muK0_for_given_Pi_ms(...)` solves for the equilibrated far-right quark state Q at $\mu_K = 0$ using the `ms`-aware momentum-flux helper throughout.
3. `_microphysics_from_quark_state(...)` computes the local coefficients D, η, γ , and τ from the local thermodynamic state, with $\mu_Q = \mu_B/3$ and the local temperature.
4. `_quark_state_entropy_residual(...)` and `_quark_state_entropy_residual_ok(...)` define the three-equation entropy closure residual and the acceptance test for a local root solve.
5. `_solve_quark_entropy_state_once_from_guess(...)` carries out one nonlinear solve in $(\mu_B, \mu_K, \log T)$ from a single continuation guess.
6. `_solve_local_quark_state_from_a_w_and_Pi(...)` wraps the previous helper into the local closure used inside the BVP. It first tries nearest-neighbor continuation, then falls back to a richer seed set on the same physical positive- μ_K branch.
7. `_seed_entropy_jB_guess(...)` obtains a robust initial guess for j_B by reusing the current non-entropy compact BVP solver on a small- a_{Q^*} seed problem.
8. `_solve_steady_front_entropy_once(...)` constructs the endpoint states, solves the direct entropy-enabled BVP once, and optionally reconstructs the profile arrays.
9. `solve_steady_front_entropy(...)` is the public wrapper. It first tries the direct one-shot solve, and if that fails, it switches to staged continuation in a_{Q^*} while recycling the last converged entropy profile as the next initial guess.

The small utility `_strip_entropy_profile_fields(...)` only trims profile arrays from returned dictionaries when the wrapper does not need them; it is a return-shaping helper, not a physics helper.

7.3 Thermodynamic closure and local state reconstruction

At fixed j_B , Π , a , and w , the local entropy-enabled quark state is reconstructed by solving for $(\mu_B, \mu_K, \log T)$ rather than (μ_B, μ_K, T) directly. The closure equations are

$$\Pi_{QM}(\mu_B, \mu_K, T; j_B) - \Pi = 0, \quad (42)$$

$$\frac{n_K(\mu_B, \mu_K, T) - n_K^Q}{n_B^Q} - a = 0, \quad (43)$$

$$s_{QM}(\mu_B, \mu_K, T) \frac{j_B}{n_B^{QM}(\mu_B, \mu_K, T)} - w = 0. \quad (44)$$

Here n_B^Q and n_K^Q come from the asymptotic equilibrated state Q .

The entropy density is not computed by a separate EOS entry point. Instead the code reconstructs it from the thermal relation

$$s = \frac{e + P - \mu_B n_B - \mu_K n_K}{T}, \quad (45)$$

using the existing quark EOS helpers for e , P , n_B , and n_K . This keeps the entropy-enabled path fully **ms**-consistent.

The entropy solver fails fast on nonphysical local states. In particular, the new helper chain rejects $T \leq 0$, $w \leq 0$, non-positive n_B , negative μ_K beyond numerical tolerance, non-positive entropy density, and local root solves that do not satisfy the momentum, composition, and entropy-flux constraints to the required tolerance.

7.4 Local microphysics and compact-coordinate BVP

The compact coordinate now uses the code name **s_coord** to avoid confusion with the entropy density s :

$$s_{\text{coord}} = 1 - e^{-x/\kappa}, \quad x(s_{\text{coord}}) = -\kappa \log(1 - s_{\text{coord}}), \quad \frac{dx}{ds_{\text{coord}}} = \frac{\kappa}{1 - s_{\text{coord}}}. \quad (46)$$

The entropy-enabled profile evolves the first-order system

$$\frac{da}{dx} = \frac{q + ua}{D(x)}, \quad (47)$$

$$\frac{dq}{dx} = \gamma(x)(a^3 + \eta(x)a), \quad (48)$$

$$\frac{dw}{dx} = \frac{n_B^Q}{T(x)} \mu_K(x) \gamma(x) (a^3 + \eta(x)a), \quad (49)$$

with $w = s u$ and

$$u(x) = \frac{j_B}{n_B(x)}. \quad (50)$$

After the compact-coordinate change this becomes

$$\frac{da}{ds_{\text{coord}}} = \frac{q + ua}{D(x)} \frac{\kappa}{1 - s_{\text{coord}}}, \quad (51)$$

$$\frac{dq}{ds_{\text{coord}}} = \gamma(x)(a^3 + \eta(x)a) \frac{\kappa}{1 - s_{\text{coord}}}, \quad (52)$$

$$\frac{dw}{ds_{\text{coord}}} = \frac{n_B^Q}{T(x)} \mu_K(x) \gamma(x) (a^3 + \eta(x)a) \frac{\kappa}{1 - s_{\text{coord}}}. \quad (53)$$

Unlike the older compact BVP, the coefficients

$$D(x), \quad \eta(x), \quad \gamma(x), \quad \tau(x) \quad (54)$$

are evaluated pointwise from the local state through `_microphysics_from_quark_state(...)`.

The entropy-enabled BVP has three first-order fields and one unknown scalar parameter, so it uses four boundary conditions:

$$a(0) - a_{Q^*} = 0, \quad (55)$$

$$q(0) + a_N u_N = 0, \quad (56)$$

$$w(0) - s_{Q^*} u_{Q^*} = 0, \quad (57)$$

$$q(s_{\text{end}}) + (D_Q \lambda + u_Q) a(s_{\text{end}}) = 0. \quad (58)$$

The first two conditions are the same composition and interface-flux constraints as before. The third enforces interface entropy-flux continuity on the quark side, using the interface state Q^* at the input temperature. The fourth is the downstream stable-tail condition. In the current implementation the tail quantities are built from the asymptotic equilibrated state Q :

$$u_Q = \frac{j_B}{n_B^Q}, \quad (59)$$

$$\lambda = \frac{-u_Q + \sqrt{u_Q^2 + 4D_Q \gamma_Q \eta_Q}}{2D_Q}, \quad (60)$$

where D_Q , η_Q , and γ_Q are evaluated at Q , not at Q^* .

7.5 One-shot solver, public wrapper, and returned data

The public entropy solver has the interface

```
solve_steady_front_entropy(
    T, nB_N, B_one_forth, aQstar,
    ms=0.0, param=para.paraQMCRMF3, NM_type="PNM",
    tail_eps=1e-8, n_mesh=200, tol_bvp=1e-4, max_nodes=10000,
    jB_guess=None, jB_bounds=None, return_profile=False, verb=False,
)
```

Its leading arguments mirror the older compact BVP path: upstream thermodynamic input, bag parameter, and the fixed interface composition a_{Q^*} . If `return_profile=True`, the returned dictionary also includes the profile arrays `s_coord`, `x`, `a`, `q`, `w`, `u`, `nB`, `muB`, `muK`, `T_profile`, `s_profile`, `D_profile`, `eta_profile`, and `gamma_profile`.

At minimum, the returned scalar diagnostics include the endpoint-state fields `success`, `message`, `jB`, `aQstar`, `a_N`, `u_N`, `u_Q`, `Pi`, `muB_Q`, `nB_Q`, `nK_Q`, `muB_Qstar`, `muK_Qstar`, `nB_Qstar`, and `s_Qstar`; the asymptotic-tail and microphysics fields `D_Q`, `eta_Q`, `gamma_Q`, `tau_Q`, `lambda`, `kappa`, `s_end`, `x_end`, `tail_residual`, and `tail_residual_norm`; and the BVP diagnostics `bvp_status`, `bvp_message`, `bvp_niter`, and `bvp_nodes`. When continuation is used, the wrapper also returns `continuation_used`, `continuation_steps`, `continuation_seed_aQstar`, and `continuation_refined`.

The helper `_solve_steady_front_entropy_once(...)` is the direct one-shot BVP solver. The public wrapper `solve_steady_front_entropy(...)` first tries that direct path. If it fails, the wrapper seeds j_B from a small- a_{Q^*} non-entropy BVP, solves a sequence of coarse continuation stages in a_{Q^*} while recycling the last converged profile, and finally retries the full target solve with the requested mesh and tolerance. This entropy-enabled public solver follows only the physical positive- μ_K branch. The compatibility field `branch_label` is still reported as "muK-rich".

Current limits. The continuation and seeding logic substantially improve robustness, but the current entropy-enabled solver should still be treated as a first implementation. For sufficiently large a_{Q^*} the continuation chain can still fail because the local thermodynamic closure or the BVP collocation step leaves the physical positive- μ_K branch. The manual therefore documents the present behavior as implemented, not a guarantee of universal convergence.

8 Practical Notes

8.1 Physical branch convention

The current steady-front solvers in `phase_velocity.py` do not expose a separate branch switch. Instead they seed and rank roots so that the solution stays on the physical positive- μ_K branch. Where result dictionaries still carry `branch_label`, the value "muK-rich" should be read as a legacy compatibility label rather than a live branch-selection control.

8.2 $m_s = 0$ convention at the far-right state

At the downstream equilibrated state Q , the code now sets

$$n_K^Q = 0$$

exactly whenever $|m_s| \leq 10^{-12}$. This is consistent with the intended symmetry of the massless-flavor limit at $\mu_K = 0$.

8.3 Recommended scan usage

If the user wants a one-dimensional solution family over the interval $a_{Q^*} \in [0, 1]$, the intended pattern is:

$$\text{aQstar_values} = \text{np.linspace}(0.0, 1.0, N). \quad (61)$$

The curve-extraction helper will then carry out continuation in j_B .

9 Summary

The update to `phase_velocity.py` introduced one-parameter steady-front solvers built around the asymptotic stable-tail condition, including both non-entropy and entropy-enabled compact-coordinate BVP paths. The current implementation:

- keeps the existing 2D solver intact,
- reuses the same endpoint-state and local-state construction logic,
- uses positive- μ_K seeding and candidate ranking to stay on the physical branch,
- solves a scalar equation for j_B at fixed a_{Q^*} ,
- adds a compact-coordinate 1D variant for long-tail integrations,
- adds a non-entropy compact-coordinate BVP variant that solves the profile and j_B together with `solve_bvp`,

- adds an entropy-enabled compact-coordinate BVP variant with local closure in $(\mu_B, \mu_K, \log T)$ and entropy flux $w = s u$ as a third ODE field,
- updates the quark-side microphysics to be evaluated locally along the entropy-enabled profile rather than frozen once at Q^* ,
- wraps the public entropy solver in seeding and continuation logic so it can retry difficult targets with recycled profile guesses,
- provides a dedicated helper to extract the numerical curve $j_B(a_{Q^*})$ efficiently.